PlantVillage Disease Detection

Team members:

1. Norhan Medhat

2. Hemmat Hamdi

3. Noran Mohammed

4. Nada Mahmoud

5. Mina Saber

6. Yousef Emad

1. Executive Summary

This document provides the complete technical documentation for a deep-learning project aimed at binary classification of plant leaf images as either healthy or diseased. The project leverages the publicly available PlantVillage dataset and implements two neural-network architectures — a Convolutional Neural Network (CNN) and an Artificial Neural Network (ANN) — to compare their performance on the same classification task.

The project was developed and executed in Google Colab, with trained model weights persisted to Google Drive. All experiments use the color subset of the PlantVillage dataset, resized to 128×128 pixels and served via Keras ImageDataGenerators to manage memory efficiently.

Attribute	Details
Project Name	PlantVillage Binary Disease Classifier
Dataset	PlantVillage (Kaggle: abdallahalidev/plantvillage-dataset)
Task	Binary image classification: Healthy vs. Diseased
Models	CNN (Conv2D) and ANN (Dense/Flatten)
Input Shape	128 × 128 × 3 (RGB)
Framework	TensorFlow 2.x / Keras
Platform	Google Colab + Google Drive
Primary Metric	Validation & Test Accuracy

2. Dataset

2.1 Source & Description

The PlantVillage dataset is a large-scale, publicly available benchmark for plant disease recognition. It contains colour photographs of healthy and diseased crop leaves spanning 38 disease categories across 14 crop species.

- Kaggle identifier: abdallahalidev/plantvillage-dataset
- Subset used: color (RGB JPEG images)
- Class grouping: folders containing 'healthy' form the positive class; all others form the diseased class.

2.2 Class Distribution & Balancing

Because the healthy and diseased classes are naturally imbalanced, a random under-sampling strategy is applied so both classes contribute equally to training.

```
min_size       = len(df_healthy)
df_diseased_bal = df_diseased.sample(n=min_size, random_state=42)
df_balanced    = pd.concat([df_healthy, df_diseased_bal]).sample(frac=1, random_state=42)
```

2.3 Data Splitting

The balanced DataFrame is split into three non-overlapping stratified subsets:

Split	Proportion	Purpose
Train	68 %	Model weight optimisation
Validation	12 %	Hyperparameter tuning & early stopping
Test	20 %	Final, unbiased performance evaluation

3. Data Pipeline

3.1 ImageDataGenerator Configuration

Keras ImageDataGenerators load images in batches directly from disk. For this experiment, only pixel normalisation is applied (dividing by 255 to map values to [0, 1]).

```
train_datagen = ImageDataGenerator(rescale=1./255)
val_datagen   = ImageDataGenerator(rescale=1./255)
test_datagen  = ImageDataGenerator(rescale=1./255)
```

3.2 Generator Parameters

Parameter	Value / Rationale
target_size	(128, 128) — balances spatial resolution with computational cost
batch_size	32 — standard mini-batch; fits in GPU/TPU memory comfortably
class_mode	'binary' — single sigmoid output; labels mapped to 0/1
shuffle (train)	True — randomises order each epoch to reduce overfitting
shuffle (val/test)	False — preserves label alignment for metric computation

4. CNN Architecture

4.1 Design Rationale

The primary model is a custom Convolutional Neural Network designed to extract hierarchical spatial features from plant leaf images. The architecture follows progressively deeper convolutional blocks, each followed by batch normalisation, max-pooling, and dropout for regularisation.

4.2 Layer-by-Layer Specification

Layer / Block	Key Parameters	Output Shape	Purpose
Conv2D (32 filters)	3×3, same, ReLU	128×128×32	Edge & colour detection
BatchNormalization	—	128×128×32	Stabilise activations
Conv2D (32 filters)	3×3, valid, ReLU	126×126×32	Refine features
MaxPooling2D	2×2	63×63×32	Spatial downsampling
Dropout (25 %)	—	63×63×32	Regularisation
Conv2D (64 filters)	3×3, same, ReLU	63×63×64	Mid-level features
BatchNormalization	—	63×63×64	Stabilise activations
Conv2D (64 filters)	3×3, valid, ReLU	61×61×64	Refine features
MaxPooling2D	2×2	30×30×64	Spatial downsampling
Dropout (25 %)	—	30×30×64	Regularisation
Conv2D (128 filters)	3×3, same, ReLU	30×30×128	High-level textures
BatchNormalization	—	30×30×128	Stabilise activations
MaxPooling2D	2×2	15×15×128	Spatial downsampling
Dropout (30 %)	—	15×15×128	Regularisation
GlobalAveragePooling2D	—	128	Compact representation
Dense (128)	ReLU	128	Non-linear classification
Dropout (50 %)	—	128	Strong regularisation
Dense (1)	Sigmoid	1	Binary probability output

4.3 Compilation

```
model.compile(
    optimizer = 'adam',
    loss      = 'binary_crossentropy',
    metrics   = ['accuracy']
)
```

5. ANN Architecture

5.1 Design Rationale

An Artificial Neural Network is implemented as a baseline comparator. Unlike the CNN, the ANN treats each pixel value as an independent feature after flattening the 128×128×3 input into a 49,152-dimensional vector, quantifying the benefit of convolutional feature extraction.

5.2 Layer Specification

Layer	Configuration	Output Size
Flatten	Input: (128, 128, 3)	49,152
Dense + BN + Dropout	512 units, ReLU, Dropout 40 %	512
Dense + BN + Dropout	256 units, ReLU, Dropout 40 %	256
Dense + BN + Dropout	128 units, ReLU, Dropout 30 %	128
Dense + Dropout	64 units, ReLU, Dropout 30 %	64
Dense (output)	1 unit, Sigmoid	1

5.3 Hyperparameter Search

A grid search was conducted over learning rates [0.001, 0.0005, 0.0001] and epoch counts [5, 10] using a simpler 2-layer ANN variant to identify the best starting configuration before training the final deep ANN.

6. Training Procedure

6.1 Callbacks

Callback	Configuration & Purpose
ModelCheckpoint	monitor: val_accuracy | save_best_only: True | Saves the epoch with the highest validation accuracy to disk.
EarlyStopping	monitor: val_accuracy | patience: 5 | restore_best_weights: True | Halts training if val_accuracy does not improve for 5 consecutive epochs.
ReduceLROnPlateau	monitor: val_loss | factor: 0.5 | patience: 3 | min_lr: 1e-6 | Halves the learning rate when validation loss plateaus.

6.2 Training Configuration Summary

Parameter	CNN	ANN
Optimizer	Adam (default lr)	Adam (lr = 0.001)
Loss	Binary Crossentropy	Binary Crossentropy
Max Epochs	5 (+ early stopping)	6 (+ early stopping)
Batch Size	32	32
Input Size	128 × 128 × 3	128 × 128 × 3 (flattened)
Model Save Path	/content/drive/MyDrive/best_model.h5	best_ann_model.h5

7. Evaluation & Results

7.1 Metrics Reported

- Test Loss — binary cross-entropy on the held-out test set
- Test Accuracy — proportion of correctly classified images
- Classification Report — per-class precision, recall, and F1-score
- Confusion Matrix — true/false positive and negative counts visualised as a heatmap

7.2 Training Curve Analysis

Both models log accuracy and loss for every epoch on training and validation sets. These are plotted as dual line charts to detect overfitting, underfitting, or convergence issues.

7.3 Model Comparison

```
=== Model Comparison on Test Set ===
Model       Loss   Accuracy
---------------------------------
CNN       <loss>   <accuracy>
ANN       <loss>   <accuracy>
```

8. Dependencies & Environment

Library	Role
tensorflow / keras	Model building, training, and inference
numpy	Numerical array operations
pandas	DataFrame management for image paths and labels
opencv-python (cv2)	Image loading and colour-space conversion
matplotlib / seaborn	Training curve and confusion matrix plots
scikit-learn	train_test_split, classification_report, confusion_matrix
tqdm	Progress bars during dataset traversal
kagglehub	Programmatic dataset download from Kaggle
glob	File path pattern matching

8.1 Execution Environment

- Runtime: Google Colab (Python 3.x)
- Accelerator: GPU (recommended for CNN training)

- Storage: Google Drive mounted at /content/drive for model persistence
- Dataset storage: Kaggle cache managed by kagglehub

9. Notebook Structure

The Jupyter notebook is organised into the following sequential sections:

#	Section	Description
1	Imports	All library imports and dependency installation
2	Dataset Download	kagglehub download and path resolution
3	Data Exploration	Folder traversal, healthy/diseased collection, DataFrame creation
4	Dataset Balancing	Under-sampling diseased class to match healthy count
5	Sample Visualisation	Random sample grid of healthy and diseased images
6	Data Splitting	Stratified train / validation / test split
7	ImageDataGenerator Setup	Batch generators with rescaling for all splits
8	Generator Verification	Batch shape checks and sample visualisation
9	CNN Model Definition	Sequential Conv2D architecture with BN and Dropout
10	CNN Compilation	Adam optimiser, binary cross-entropy loss
11	CNN Callbacks	Checkpoint, EarlyStopping, ReduceLROnPlateau
12	CNN Training	model.fit() with callbacks
13	CNN Evaluation	Test loss and accuracy; learning curves
14	ANN Generators	Separate generators for ANN experiments
15	ANN Hyperparameter Search	Grid search over lr and epoch counts
16	Final ANN Model	Deep ANN with 4 hidden layers
17	ANN Training	ann_model.fit() with callbacks
18	ANN Evaluation	Confusion matrix and classification report
19	Model Comparison	Side-by-side CNN vs. ANN test results

10. Reproduction Guide

Step 1: Environment Setup

- Open the notebook in Google Colab
- Mount Google Drive when prompted (required for model saving)
- Enable a GPU runtime: Runtime → Change runtime type → T4 GPU

Step 2: Dataset

- Ensure a valid Kaggle API token is configured (kaggle.json) or use kagglehub authentication
- Run the download cell — the dataset is cached by kagglehub on subsequent runs

Step 3: Execute All Cells

- Run all cells top to bottom
- Training will automatically stop early if val_accuracy stops improving
- Best weights are restored automatically by EarlyStopping

Step 4: Review Results

- Training curves are plotted inline
- Test accuracy, loss, classification report, and confusion matrix are printed/displayed
- The final comparison table shows CNN vs. ANN head-to-head

11. Limitations & Future Work

11.1 Current Limitations

- Binary classification only: the model distinguishes healthy from diseased but does not identify specific diseases or crop species.
- Limited augmentation: only pixel rescaling is applied; no rotations, flips, or colour jitter are used.
- Short training schedule: the 5-epoch cap prioritises fast iteration over peak performance.
- No transfer learning: a pre-trained backbone (e.g., EfficientNet, ResNet) would likely achieve higher accuracy with fewer epochs.

11.2 Recommended Next Steps

- Extend to multi-class classification across all 38 PlantVillage disease categories.
- Apply data augmentation (random flips, rotations, zoom, colour jitter) to improve robustness.
- Integrate transfer learning using a pre-trained ImageNet backbone.
- Deploy the trained model as a mobile or web application for on-field diagnosis.
- Evaluate on external datasets (field-captured images) to measure real-world performance.